Go language fundamentals

Go is a procedural language with object-oriented and functional features.

It doesn't fit neatly into a single box because it was designed to be pragmatic rather than dogmatic. It takes the best parts of various paradigms to make code easy to maintain.

Go is MULTI-PARADIGM — primarily procedural, with object-oriented FEATURES (not pure OOP), and some functional programming support.

Go is NOT:

✗ Pure Object-Oriented (no classes, no inheritance)
✗ Pure Functional (no immutability, no pure functions enforced)
✗ Pure Procedural (has methods, interfaces, closures)

Go IS:

✓ Procedural at its core
✓ OOP-like via structs + methods + interfaces
✓ Functional-ish via first-class functions + closures

1. Go as a Procedural Language

At its core, Go is procedural. It follows a top-down execution flow where the focus is on **functions** and **data structures**.

- Programs are organized into packages.
- It uses clear, imperative statements (if/else, for-loops).
- There is no "hidden" state like you find in heavy class-based languages.

2. Go as an Object-Oriented (OO) Language

Go is "OO" in spirit, but not in syntax. It lacks classes and inheritance, which are the hallmarks of traditional OOP (like Java or C#).

- **Encapsulation:** Go uses **structs** to group data and **methods** to define behavior for those structs.
- **Visibility:** Instead of public or private keywords, Go uses **Capitalization**. MyFunction is public; myFunction is private.
- **Polymorphism via Interfaces:** This is Go's "superpower." Interfaces are satisfied implicitly. If a struct has the required methods, it "is" that interface. You don't need an implements keyword.
- **No Inheritance:** Go prefers **Composition**. You "embed" one struct into another to reuse code, rather than building a rigid family tree.

OOP Concept	Classical OOP (Java / C++)	Go Implementation
Object Definition	class Animal {}	type Animal struct {}
Inheritance	extends, inherits	Struct embedding (composition)
Interface Implementation	implements keyword	Implicit interface implementation
Methods	Methods defined inside classes	Methods can be defined on any type
Constructor	new or constructor methods	Factory functions
Abstract Class	abstract class	interface
Method Overriding	Override methods in subclasses	Interface satisfaction / method implementation
Object Reference	this / self keyword	Receiver (a Animal)

What Go OOP is Missing (and Alternatives)

Missing Feature in Go	Alternative in Go	Explanation
class keyword	struct + methods	Go uses structs to define data and methods to define behavior
Inheritance (extends)	Embedding (composition)	One struct can embed another struct to reuse fields and methods
Method overriding	Interface re-implementation	Different types implement the same interface method
Constructors	NewXxx() factory functions	Go uses functions to create and initialize objects
Access modifiers (private/public)	Uppercase / lowercase naming	Capitalized names are exported, lowercase are package-private
Abstract classes	Interfaces	Interfaces define behavior without implementation
this / self keyword	Named receiver (p Person)	Methods explicitly declare the receiver
Operator overloading	Not supported	Go intentionally avoids operator overloading
Generics (before Go 1.18)	Generics added in Go 1.18	Type parameters allow reusable code

3. Go as a Functional Language

Go is not a "pure" functional language (like Haskell), but it treats functions as **first-class citizens**.

- **Higher-Order Functions:** You can pass functions as arguments to other functions or return them.
- **Anonymous Functions & Closures:** You can define functions on the fly within other functions.
- **No "Map/Filter/Reduce" in the core syntax:** While you can implement these, Go encourages using simple for loops for clarity and performance, which is a departure from functional purity.

The Verdict: A Hybrid Approach

Go is often called a **post-OOP language**. It provides the benefits of objects (modularity and abstraction) without the complexity of deep inheritance chains.

Feature	Is it in Go?	Notes
Methods	Yes	Attached to structs or any local type.
Classes	No	Structs fill this role.
Inheritance	No	Replaced by Composition/Embedding.
Pure Functions	Optional	Functions can have side effects (and usually do).
State	Yes	Encourages mutable state within clear boundaries.

1. Program Structure

Every Go source file follows a rigid structural pattern. This consistency is why Go code is famously easy to read, even if you didn't write it.

- **Package Declaration:** Every file must start with package <name>.
- **Import Statements:** Used to include code from other packages.
- **Actual Code:** Functions, variables, types, and constants.

2. The Entry Point

In Go, the execution starts in a specific place. For a compiler to produce an executable binary, two conditions must be met:

1. The file must be part of **package main**.
2. It must contain a function named **func main()**.

Eg

```
“main.go”
package main

import "fmt"

func main() {
  fmt.Println("Hello, Gopher!")
}
```

Note: The main function takes no arguments and returns no values. If you need to handle command-line arguments, you use the os package (e.g., os.Args).

3. The Type System
Go's type system is **static** (checked at compile time) and **strong** (no implicit conversions). It is designed for safety and efficiency without the "bloat" of class hierarchies.

Composition over Inheritance
Go does not have classes or inheritance. Instead, it uses **structs** and **interfaces**. You extend functionality through "embedding" rather than "subclassing."

Key Type Categories:

- **Basic Types:** bool, string, int, float64, complex128.
- **Composite Types:** * **Structs:** Collections of fields.
    - **Arrays & Slices:** Slices are the flexible, dynamic version of arrays.
    - **Maps:** Key-value pairs (hash tables).
- **Interfaces:** These define behavior. If a type implements the methods required by an interface, it satisfies that interface automatically (**duck typing**).

Static vs. Dynamic Typing

While Go is statically typed, it offers shorthand for type inference using the short declaration operator:

- var x int = 10 (Explicit)
- x := 10 (Inferred as int)

Summary of Key Rules

- **Unused items are errors:** If you import a package or declare a variable and don't use it, the program will not compile. This keeps the codebase lean.

- **Capitalization matters:** If a function or variable starts with an uppercase letter (e.g., Print), it is **exported** (public). If it's lowercase, it's **unexported** (private to the package)
- **Semi colon: In Go,** the semicolon (;) is the "invisible worker." If you are coming from C, C++, or Java, you're used to typing them at the end of every line. In Go, you almost never see them, but they are still there.
- Here is how Go handles semicolons and why they matter for the language's structure.

1. The "Lexical Insertion" Rule

Go uses a **Semicolon Insertion** rule. The lexer (the part of the compiler that reads your code) automatically injects semicolons at the end of a line if the last "token" on that line is:

- An identifier (like a variable name).
- A basic literal (like a number or string).
- A closing bracket or parenthesis (),], or }).
- Certain keywords (like break, continue, fallthrough, or return).

Because of this, you don't have to type them yourself. The compiler does the "busy work" for you to keep the code clean.

2. The "Brace" Trap

Because the compiler inserts semicolons automatically, Go has a **strict rule** about where you place your curly braces {.

The Correct Way:

```
func main() {  // The brace must be on the same line
```

```
  fmt.Println("Success")
}
```

The Wrong Way (Will fail to compile):

```
func main()
{  // The lexer puts a ';' after main(), breaking the code!
  fmt.Println("Fail")
}
```

In the "Wrong" example, the compiler sees func main(), notices the closing parenthesis, and inserts a semicolon. This makes the compiler think the function definition ended right there, leading to a syntax error.

3. When You *Do* Use Semicolons

There are three specific places where you will still manually type a semicolon in Go:

A. The Three-Part for Loop

Just like in C, the for loop uses semicolons to separate the initializer, condition, and post-statement.

```
for i := 0; i < 10; i++ {
  fmt.Println(i)
}
```

B. If/Switch Initialization

Go allows you to execute a short statement (like a function call or assignment) before the actual condition check. A semicolon separates the two.

Go

```
if err := saveFile(); err != nil {
   return err
}
```

C. Multiple Statements on One Line

If you absolutely must put two statements on a single line (which is generally discouraged by the Go community), you use a semicolon to separate them.

Go

```
x := 1; y := 2
```

Summary

- **Most of the time:** Don't type them.
- **The Compiler:** Injects them based on line breaks.
- **The Result:** Very strict rules about newline placement and curly braces.

Go Type system:

Category	Sub Category	Type	Example
Basic Types	Numeric	int	var a int = 10
		int8, int16, int32, int64	var x int32 = 100
		uint, uint8, uint16, uint32, uint64	var u uint = 20

	Floating	float32, float64	var price float64 = 10.5
	Complex	complex64, complex128	var c complex64 = 1+2i
	String	string	name := "Go"
	Boolean	bool	flag := true
	Byte	byte (alias for uint8)	var b byte = 255
	Rune	rune (alias for int32)	var r rune = 'A'
Composite Types	Array	[5]int	var arr [5]int
	Slice	[]int	nums := []int{1,2,3}
	Map	map[string]int	scores := map[string]int{"A":10}
	Struct	struct	type User struct {Name string}
	Channel	chan int	ch := make(chan int)
Reference Types	Pointer	*int	var p *int
	Function	func()	func greet(){}
	Interface	interface{}	var x interface{}

Group	Types
Basic Types	int, float, bool, string, byte, rune

Composite Types	array, slice, map, struct, channel
Reference Types	pointer, function, interface

Go Variable Declarations — Complete Guide

6 Ways to Declare Variables

```
// 1. var with type and value
var name string = "Alice"

// 2. var with type only (zero value)
var name string

// 3. var with value only (type inferred)
var name = "Alice"

// 4. short declaration (most common)
name := "Alice"

// 5. multiple variables
var x, y, z int = 1, 2, 3

// 6. block declaration
var (
   name string = "Alice"
   age  int    = 25
)
```

var with Type and Value

```
var name string = "Alice"
var age  int    = 25
var pi   float64 = 3.14
var flag bool    = true
```

```
// Explicit — you control both type and value
// Verbose — rarely used when value is obvious
```

var with Type Only — Zero Values

```
// Declared but not initialized → gets ZERO VALUE

var i   int      // 0
var f   float64   // 0.0
var b   bool      // false
var s   string    // ""  (empty string)
var p   *int      // nil
var sl  []int     // nil
var m   map[string]int // nil
var fn  func()    // nil
var err  error    // nil

fmt.Println(i)     // 0
fmt.Println(f)     // 0
fmt.Println(b)     // false
fmt.Println(s)     // ""
fmt.Println(p)     // <nil>


// Zero values are ALWAYS defined in Go

// No undefined/uninitialized memory like C
```

Go Data Types with Zero Value and Size (Memory Size) in a clear **table format**.

Type	Zero Value	Size (Bytes)
int8	0	1
uint8 / byte	0	1
int16	0	2

uint16	0	2
int32 / rune	0	4
uint32	0	4
int64	0	8
uint64	0	8
int	0	8 (on 64-bit) / 4 (on 32-bit)
uint	0	8 (on 64-bit) / 4 (on 32-bit)
uintptr	0	8
float32	0.0	4
float64	0.0	8
complex64	(0+0i)	8
complex128	(0+0i)	16
bool	false	1
string	""	16 (pointer + length)
pointer (*T)	nil	8
slice	nil	24 (pointer + len + cap)
map	nil	8 (pointer to runtime structure)
channel	nil	8
function	nil	8
interface	nil	16 (type + value pointer)
array	all elements zero	element size × length
struct	fields zero	sum of field sizes + padding

Signed vs Unsigned Integer Types in Go

Type	Signed / Unsigned	Size (Bytes)	Value Range
int8	Signed	1	-128 to 127
uint8 (byte)	Unsigned	1	0 to 255
int16	Signed	2	-32,768 to 32,767
uint16	Unsigned	2	0 to 65,535
int32 (rune)	Signed	4	-2,147,483,648 to 2,147,483,647
uint32	Unsigned	4	0 to 4,294,967,295
int64	Signed	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
uint64	Unsigned	8	0 to 18,446,744,073,709,551,615

Key Difference

Feature	int Types	uint Types
Negative numbers	Allowed	Not allowed
Range	Half positive / half negative	All positive
Prefix	int	uint
Use case	General arithmetic	Bit manipulation, memory, binary data

var with Value Only — Type Inferred

```
var name   = "Alice"      // string
var age    = 25           // int
var pi     = 3.14         // float64
var flag   = true         // bool
var nums   = []int{1,2,3}  // []int

// Compiler INFERS the type from the value
// Type is still FIXED — not dynamic

name = 42                 // COMPILE ERROR — name is string
```

Short Variable Declaration :=

```
// Most common way — inside functions only
name := "Alice"           // string
age  := 25                // int
pi   := 3.14              // float64
flag := true              // bool

// ✓ Concise
// ✓ Type inferred
// ✗ Only works INSIDE functions
// ✗ Cannot use at package level
```

Multiple Variable Declaration

```
// Same type
var x, y, z int           // 0, 0, 0
var a, b, c int = 1, 2, 3

// Different types — use var block
var (
```

```
    name   string  = "Alice"
    age    int     = 25
    height  float64 = 5.6
    active  bool    = true
)

// Short declaration — multiple
x, y := 10, 20
name, age := "Alice", 25

// Swap values elegantly
x, y = y, x            // x=20, y=10  (no temp variable needed)

// Ignore values with blank identifier
value, _ := someFunc()    // ignore second return value
_, err   := anotherFunc()  // ignore first return value
```

Strings:
In Go (Golang), a string is a read-only slice of bytes, usually representing UTF-8–encoded text, and is immutable once created. Strings are one of the most commonly used types for text processing.

Basic string syntax

- Declared with double quotes ("...") or raw (interpreted) literals with backticks (`...`)

Method	**Example**
Standard declaration	var s string = "Hello"
Type inference	s := "Hello"
Empty string	var s string
Raw string (multiline)	s := `Hello Go`

Strings Package

Go provides many utilities in the **strings package**.

Common functions:

Function	**Description**

strings.Contains()	Check substring
strings.HasPrefix()	Check prefix
strings.HasSuffix()	Check suffix
strings.ToUpper()	Convert to uppercase
strings.ToLower()	Convert to lowercase
strings.TrimSpace()	Remove spaces
strings.Split()	Split string
strings.Join()	Join strings
strings.Replace()	Replace substring

type Keyword in Go

In Go, the **type keyword** is used to **define new types**.
It helps you create **custom data structures**, **aliases**, and **interfaces**.

1. Purpose of type

The type keyword allows you to:

Use Case	Description
Create custom types	Define new data types
Define structs	Create structured objects
Define interfaces	Specify method behavior
Create type aliases	Alternative names for types
Define function types	Create reusable function signatures

Eg:

```
package main

import "fmt"

func main() {
   type Age int
   var a Age = 25
   fmt.Print(a)
}
```

```
type Celsius float64
var temp Celsius = 36.5
```

Operators

Go (Golang) provides a concise set of operators for arithmetic, comparison, logical, bitwise, and assignment operations, emphasizing simplicity and no operator overloading.

Arithmetic Operators

These handle basic math on numeric types like int, float64.

- + adds two values (e.g., a + b).
- - subtracts (e.g., a - b).
- * multiplies (e.g., a * b).
- / divides (e.g., a / b; integer division truncates).
- % computes remainder (e.g., a % b).
- ++ increments by 1 (postfix only, statement not expression).
- -- decrements by 1 (postfix only).

Unary + and - also exist for sign.

Assignment Operators

Combine assignment with arithmetic or bitwise ops.

Operator	Example	Equivalent
=	a = b	-
+=	a += b	a = a + b
-=	a -= b	a = a - b
***=**	a *= b	a = a * b
/=	a /= b	a = a / b
%=	a %= b	a = a % b

Comparison Operators

Return bool; work on comparable types (numbers, strings).

- == checks equality.
- != checks inequality.
- < less than.

- <= less or equal.
- > greater than.
- >= greater or equal.

Logical Operators
Operate on bools (short-circuit evaluation).
- && (AND): true if both true.
- || (OR): true if either true.
- ! (NOT): inverts bool.

Bitwise Operators
Manipulate integer bits.
- & AND.
- | OR.
- ^ XOR.
- << left shift.
- >> right shift (arithmetic for signed).
- &^ AND NOT (bits set to 0 where right is 1).

Note: No ternary operator (? :); use if-else. Operators follow strict precedence (unary highest, then * / %, etc.)

```
package main

import "fmt"

func main() {
   // Base values
   a, b, c, d := 10, 3, 15, 7
   x, y := true, false
   p := 12  // For bitwise

   // Arithmetic
   fmt.Printf("Arithmetic: %d + %d = %d, %d - %d = %d, %d * %d = %d\n",
           a, b, a+b, a, b, a-b, a, b, a*b)

   // Division & Modulus
   fmt.Printf("Division: %d / %d = %d, %d %% %d = %d\n", a, b, a/b, a, b, a%b)
```

```
    // Unary +/-
    fmt.Printf("Unary: +%d = %d, -%d = %d\n", a, +a, a, -a)

    // Increment/Decrement (statements only)
    a++  // Now 11
    b--  // Now 2
    fmt.Printf("Inc/Dec: a++ → %d, b-- → %d\n", a, b)

    // Assignment (=)
    e := 5
    fmt.Printf("Assignment: e = 5 → %d\n", e)

    // Compound Assignment (+=, etc.)
    e += 2  // Now 7
    fmt.Printf("Compound: e += 2 → %d\n", e)

    // Comparison
    fmt.Printf("Comparison: %d == %d → %t, %d > %d → %t\n",
            c, d, c==d, c, d, c>d)

    // Logical
    fmt.Printf("Logical: %t && %t → %t, %t || %t → %t, !%t → %t\n",
            x, y, x&&y, x, y, x||y, x, !x)

    // Bitwise
    fmt.Printf("Bitwise: %b & %b = %b (%d), %b | %b = %b (%d)\n",
            p, b, p&b, p&b, p, b, p|b, p|b)
    fmt.Printf("Shift: %b << 1 = %b (%d), %b >> 1 = %b (%d)\n",
            p, p<<1, p<<1, p, p>>1, p>>1)
    fmt.Printf("AND NOT: %b &^ %b = %b (%d)\n", p, 1, p&^1, p&^1)
}
```

Control Structures and Statements in Go:

Control structures determine **the flow of execution** in a Go program. They allow programs to make **decisions, repeat actions, and manage execution paths**.

---

1. Types of Control Structures in Go

Category	Control Structure	Purpose
Conditional	if, if-else, else if	Decision making
Multi-branch	switch	Multiple conditions
Looping	for	Repetition
Iteration	range	Loop over collections
Jump statements	break, continue, goto	Change execution flow
Defer statement	defer	Execute later
Select statement	select	Channel-based concurrency control

---

2. if Statement

Used for **conditional execution**.

Syntax

```
if condition {
   // code
}
```

Example:

```
package main
import "fmt"

func main() {

   age := 20

   if age >= 18 {
      fmt.Println("Adult")
   }
}
```

Output

Adult

---

3. if-else Statement

```
if condition {
   // true block
} else {
   // false block
}
```

Example:

```
num := 10

if num%2 == 0 {
   fmt.Println("Even")
} else {
   fmt.Println("Odd")
}
```

---

4. if-else if Ladder

```
score := 85

if score >= 90 {
   fmt.Println("Grade A")
} else if score >= 75 {
   fmt.Println("Grade B")
} else {
   fmt.Println("Grade C")
}
```

---

5. switch Statement

Used for **multiple condition checks**.

Syntax

```
switch expression {
case value1:
case value2:
default:
}
```

Example:

```
day := 2

switch day {
```

```
case 1:
   fmt.Println("Monday")

case 2:
   fmt.Println("Tuesday")

default:
   fmt.Println("Other day")
}
```

Output

Tuesday

---

6. Switch Without Expression

Go allows condition-based switch.

```
age := 20

switch {

case age < 18:
   fmt.Println("Minor")

case age >= 18:
   fmt.Println("Adult")
}
```

---

7. for Loop

Go has **only one looping keyword: for**.

Standard Loop

```
for i := 1; i <= 5; i++ {
   fmt.Println(i)
}
```

Output

1
2
3
4

5

---

8. for as While Loop

```
i := 1

for i <= 5 {
   fmt.Println(i)
   i++
}
```

---

9. Infinite Loop

```
for {
   fmt.Println("running")
}
```

---

10. range Loop

Used for iterating collections.

Example with slice:

```
nums := []int{10, 20, 30}

for index, value := range nums {
   fmt.Println(index, value)
}
```

Output

```
0 10
1 20
2 30
```

Example with map:

```
m := map[string]int{
   "A": 1,
   "B": 2,
}

for key, value := range m {
   fmt.Println(key, value)
}
```

---

11. break Statement

Stops the loop immediately.

Example:

```
for i := 1; i <= 5; i++ {

   if i == 3 {
      break
   }

   fmt.Println(i)
}
```

Output

```
1
2
```

---

12. continue Statement

Skips the current iteration.

Example:

```
for i := 1; i <= 5; i++ {

   if i == 3 {
      continue
   }

   fmt.Println(i)
}
```

Output

```
1
2
4
5
```

---

13. goto Statement

Transfers control to a labeled statement.

Example:

```
package main
```

```
import "fmt"

func main() {

   i := 1

start:
   fmt.Println(i)

   i++

   if i <= 3 {
      goto start
   }
}
```

Output
1
2
3

---

14. defer Statement

Schedules a function call to run **after the surrounding function finishes**.

Example:

```
package main
import "fmt"

func main() {

   defer fmt.Println("World")

   fmt.Println("Hello")
}
```

Output
Hello
World

---

15. select Statement

Used for **channel operations in concurrency**.
Example:

```
select {
case msg := <-ch1:
   fmt.Println(msg)

case msg := <-ch2:
   fmt.Println(msg)
}
```

select waits until one channel is ready.

Summary

Control Structure	Purpose
if	Conditional execution
switch	Multi-branch decision
for	Looping
range	Iteration over collections
break	Exit loop
continue	Skip iteration
goto	Jump to label
defer	Execute later
select	Channel communication